

CITS5017 Deep Learning

Topic 8: Attention Mechanisms, Transformers, and Vision Transformers

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

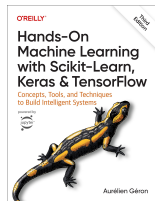
Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 16.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Natural Language Processing with RNNs and Attention

Here are the main topics we will cover:

- Encoder-decoder networks for neural machine translation
- Bidirectional RNNs and LSTMs
- Attention mechanisms, including additive attention and multiplicative attention
- Implementing attention mechanisms using Keras
- Visual attention
- The original transformer
- The original vision transformer

Encoder-Decoder Networks for Processing Sequences

In an earlier topic, we learnt that we could use an encoder-decoder network¹ for processing sequences:

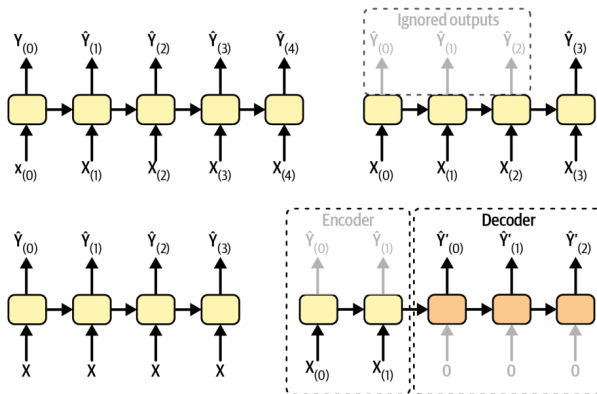


Figure 15-4. Sequence-to-sequence (top left), sequence-to-vector (top right), vector-to-sequence (bottom left), and encoder-decoder (bottom right) networks

¹Do not confuse this with the *encoder* and *decoder* in an autoencoder network.

Encoder-Decoder Networks for NMT

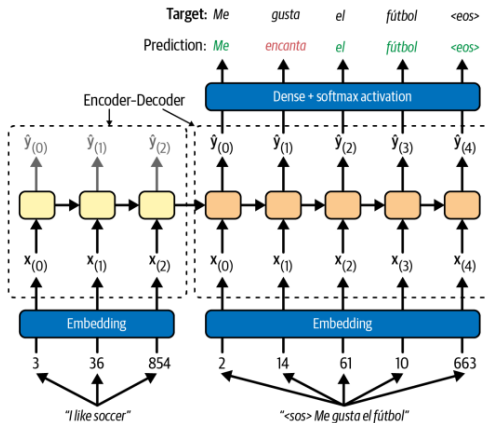


Figure 16-3. A simple machine translation model

The encoder-decoder networks are particularly suitable for *Neural Machine Translation (NMT)* problems. E.g., input: English sentences; output: Spanish sentences).

Encoder-Decoder Networks for NMT (cont.)

In NMT, words are represented as embedding vectors². The encoder and decoder parts can be implemented as a simple RNN, stacked RNN, or LSTM.

At each time step, a regular recurrent layer only looks at past and present inputs before generating its output. In other words, it is *causal*, meaning it cannot look into the future. This type of RNN makes sense when forecasting time series, or in the decoder of a sequence-to-sequence model. But for tasks like text classification and translation, it is often preferable to look ahead at the next words before encoding a given word.

e.g., consider the phrases “*the right arm*”, “*the right person*”, and “*the right to criticize*”: to properly encode the word “*right*”, you need to look ahead.

²For example, as Word2Vec, GloVe (Global Vectors for Word Representation), or fastText, etc.

Bidirectional RNNs and LSTMs

One solution is to run two recurrent layers on the same inputs, one reading the words from left to right and the other reading them from right to left, then combine their outputs at each time step, typically by concatenating them. This is what a *bidirectional recurrent layer* does:

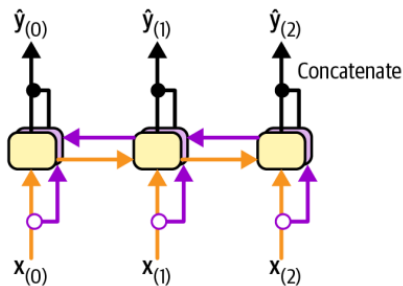


Figure 16-5. A bidirectional recurrent layer

Bidirectional RNNs and LSTMs (cont.)

For example, the following Bidirectional layer could be used as the encoder in our translation model:

```
encoder = tf.keras.layers.Bidirectional(  
    tf.keras.layers.LSTM(256, return_state=True))
```

There's just one problem. This layer will now return four states instead of two: the final short-term and long-term states of the forward LSTM layer, and the final short-term and long-term states of the backward LSTM layer. These won't fit with the decoder's LSTM layer and we cannot make the decoder bidirectional. To overcome this problem, we can concatenate the two short-term states, and also concatenate the two long-term states:

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)  
encoder_state = [  
    tf.keras.layers.Concatenate(axis=-1)(encoder_state[::2]), #short-term (0&2)  
    tf.keras.layers.Concatenate(axis=-1)(encoder_state[1::2]) #long-term (1&3)  
]
```


Bidirectional RNNs and LSTMs (cont.)

Let's try a simple Bidirectional RNN on the bus/rail dataset, with inputs being (bus, rail) values over 56 days and the outputs that we want to predict are the (bus, rail) values for the following day (57th day).

```
simple_biRNN = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=[None, 2]),
    tf.keras.layers.Bidirectional(
        tf.keras.layers.SimpleRNN(16)),
    tf.keras.layers.Dense(2)
]) # 674 trainable params
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=50, restore_best_weights=True)

# PrintEpochLossCallback -- A customized Callback class to help reduce
# the verbose printing of training loss, MAE, etc.
logging_cb = PrintEpochLossCallback(20, Nepochs)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
simple_biRNN.compile(loss="mse", optimizer=opt, metrics=["mae"])

# X_train.shape = (1027,56,2), y_train_1.shape = (1027,2)
history = simple_biRNN.fit(X_train, y_train, epochs=500,
    validation_data=(X_valid, y_valid_1),
    verbose = 0, callbacks=[logging_cb, early_stopping_cb])
```

Bidirectional RNNs and LSTMs (cont.)

```
Epoch   0: loss: 0.0932, mae: 0.2250; val_loss: 0.0221, val_mae: 0.1136
...
Epoch 160: loss: 0.0046, mae: 0.0404; val_loss: 0.0015, val_mae: 0.0279
Epoch 180: loss: 0.0046, mae: 0.0406; val_loss: 0.0017, val_mae: 0.0288
```

Now test the model on the validation set and compare with the ground truth `y_valid`:

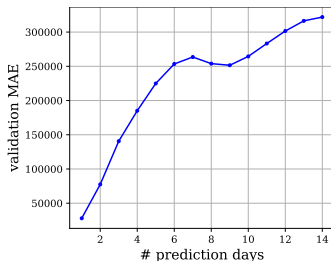
```
>>> yvalid_pred = simple_biRNN.predict(X_valid, verbose=0)
>>> print(np.mean(np.abs(yvalid_pred - y_valid)) * 1e6)
27936.60823591162
```

This MAE ([27,936](#)) is slightly worse than [27,855](#) from the RNN in topic 4 (slide [33/52](#)) where we used [\(bus, rail, one-hot-encoding\(days\)\)](#) (i.e., 5 input features) to predict the [\(bus, rail\)](#) value on the [57th](#) day.

If we include additional information from the days of the weeks, then the validation MAE should be even lower.

Bidirectional RNNs and LSTMs (cont.)

To predict more days ahead, we could use a *for* loop as shown on slide 34/52³ in Topic 4 (*Processing Sequences Using RNNs and CNNs*).

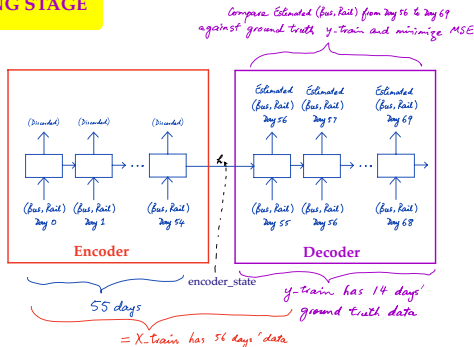


(`X_valid` has 56 days, then 57 days, then 58 days, ... in the *for* loop)
Unfortunately, the prediction errors accumulated quickly for this `simple_biRNN` model as we increase the *prediction horizon*.

³Note that the example shown on that slide is for one instance only. Modification will be needed to make it work for the whole test set.

An Encoder-Decoder Network for the Bus/Rail Dataset

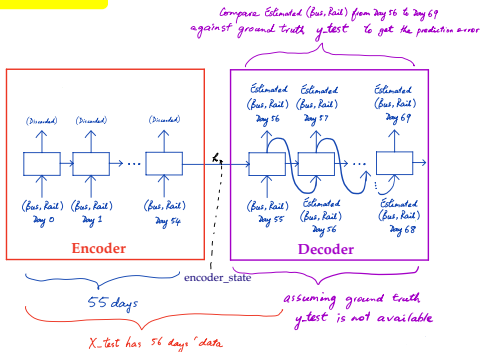
TRAINING STAGE



In this network, the *Encoder* tries to learn the patterns in the numbers of bus and rail riders over 55 days (rather than 56 days) while the *Decoder* tries to predict these numbers 14 days ahead. At the training stage, the Decoder's inputs comprise of the last day from X_{train} and y_{train} shifted by one timestep, while its outputs are just y_{train} .

An Encoder-Decoder Network for the Bus/Rail Dataset

TESTING STAGE



At the testing stage, the Decoder only has input at the **1st** timestep, coming from the last time step (day 55) of **X_test**, (i.e., **X_test[:, -1, :]**), while its inputs for other timesteps

are unknown (so we set them to **0**). The whole network is run **14** times for the **14** days' prediction. That is, the `model.predict(...)` method is called **14** times through a *for* loop. Each time in the loop when a new **(bus, rail)** value is predicted, it replaces the Decoder's input for the next day.

An Encoder-Decoder Network for the Bus/Rail Dataset

```
inputs_dim = 2    # bus and rail
# the encoder takes in (bus,rail) sequences (55 days)
encoder_inputs = tf.keras.Input(shape=[None, inputs_dim])
encoder = tf.keras.layers.Bidirectional(
    tf.keras.layers.SimpleRNN(16, return_state=True))
encoder_outputs, *encoder_state = encoder(encoder_inputs)

# encoder_state is a list containing 2 elements: the forward
# state and the backward state. We need to pass them to the decoder.
encoder_state = tf.keras.layers.Concatenate(axis=-1)(encoder_state)
# encoder_state is now a Keras tensor of shape (None,32)

# the decoder learns to predict (bus,rail) sequences (14 days
# after the input 56 days)
decoder_inputs = tf.keras.layers.Input(shape=[None, 2])

# Note: the #neurons for the decoder must match the shape of encoder_state
decoder = tf.keras.layers.SimpleRNN(32, return_sequences=True)
decoder_outputs = decoder(decoder_inputs, initial_state=encoder_state)

outputs = tf.keras.layers.Dense(2)(decoder_outputs)

rnn_ed = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                        outputs=outputs)
```

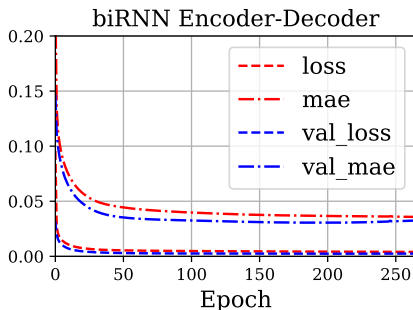
An Encoder-Decoder Network for the Bus/Rail Dataset

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss', patience=50, restore_best_weights=True)  
  
logging_cb = PrintEpochLossCallback(20, Nepochs)  
  
rnn_ed.compile(loss="mse", optimizer="nadam", metrics=["mae"])  
history = rnn_ed.fit((X_train, X_train_decode), y_train, epochs=500,  
    validation_data=((X_valid, X_valid_decode), y_valid),  
    verbose=0, callbacks=[logging_cb, early_stopping_cb])  
  
Epoch    0: loss: 0.0956, mae: 0.2332; val_loss: 0.0316, val_mae: 0.1468  
...  
Epoch 220: loss: 0.0042, mae: 0.0364; val_loss: 0.0024, val_mae: 0.0307  
Epoch 240: loss: 0.0041, mae: 0.0361; val_loss: 0.0024, val_mae: 0.0314  
Epoch 260: loss: 0.0040, mae: 0.0358; val_loss: 0.0025, val_mae: 0.0322
```

An Encoder-Decoder Network for the Bus/Rail Dataset

Inspect the learning curves to ensure there is no overfitting or underfitting issue.

```
hist = history.history
N_training_epochs = len(hist['loss'])
pd.DataFrame(hist).plot(
    figsize=(4,3), xlim=[0, N_training_epochs-1], ylim=[0, 0.2],
    grid=True, xlabel="Epoch", style=["r--", "r-.", "b--", "b-."])
plt.legend(loc="best")
plt.title('Learning curves')
plt.show()
```



An Encoder-Decoder Network for the Bus/Rail Dataset

```
def predict(model, encoder_inputs, decoder_input, outseq_len, **kwargs):
    # encoder_inputs could be X_val or X_test over 55 days (0 to 54).
    # decoder_input is X_val for 1 day only (day 55).
    # outseq_len is the required length of the output sequences (14 here).
    batch_size = encoder_inputs.shape[0]
    out_dim = decoder_input.shape[-1]

    # the output sequence to be predicted
    outseq = np.zeros((batch_size, outseq_len, out_dim))
    # construct the inputs for the decoder
    decoder_ins = np.zeros((batch_size, outseq_len, out_dim))
    decoder_ins[:, 0, :] = decoder_input
    for t in range(outseq_len):
        # call model.predict(...) and update decoder_ins and outseq
    return outseq
>>> yvalid_pred = predict(model, X_valid, X_valid_decode[:,0],
                          out_seq_length, verbose=0)
>>> print(r'Validation set MAE = {:11,.4f}'.format(
        np.mean(np.abs(yvalid_pred-y_valid))*1e6))
Validation set MAE = 34,863.8857
```

This is the lowest validation MAE so far for multi-task predictions for 14 days ahead.

Attention Mechanisms

Coming back to the Encoder-Decoder networks for NMT, consider the path from the word “soccer” to its translation “fútbol” in Figure 16-3: it is quite long! This means that a representation of this word (along with all needs to be carried over many steps before it is actually used. Can’t we make this path shorter?

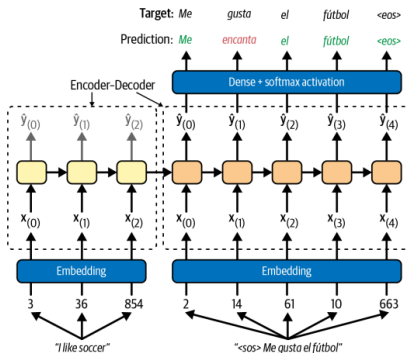


Figure 16-3. A simple machine translation model

Attention Mechanisms

In a landmark 2014 paper by Bahdanau et al.⁴, a technique was introduced that allowed the decoder to focus on the appropriate words (as encoded by the encoder) at each time step. e.g., at the time step where the decoder needs to output the word “fútbol”, it will focus its attention on the word “soccer”. This means that the path from an input word to its translation is now much shorter, so the short-term memory limitations of RNNs have much less impact. *Attention mechanisms* revolutionized neural machine translation (and deep learning in general), allowing a significant improvement in the state of the art, especially for long sentences (e.g., over 30 words).

⁴Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate”, arXiv preprint arXiv:1409.0473 (2014)

Attention Mechanisms (cont.)

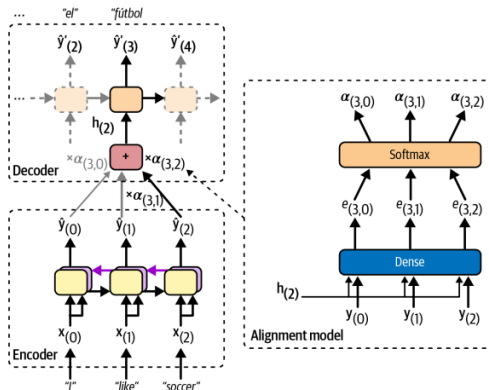


Figure 16-7. Neural machine translation using an encoder-decoder network with an attention model

Instead of just sending the **encoder's final hidden state** to the decoder, as well as the **previous target word** (not shown in the figure) at each step, **all of the encoder's outputs** are now sent to the decoder as well.

Since the decoder cannot deal with all these outputs, we need to aggregate them:

At each time step, the decoder's memory cell computes a **weighted sum** of all the encoder outputs. This determines which words it will focus on at this step.

Attention Mechanisms (cont.)

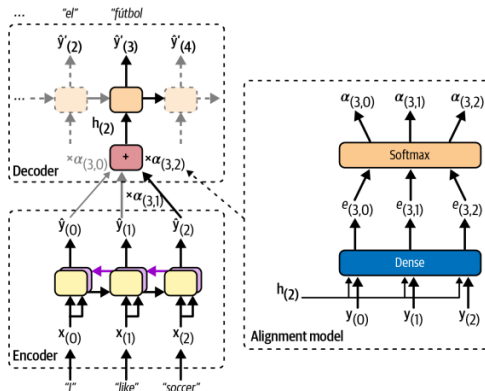


Figure 16-7. Neural machine translation using an encoder-decoder network with an attention model

The weight $\alpha(t, i)$ is the weight of the i^{th} encoder output at the t^{th} decoder time step. e.g., if, at time step 3, the weight $\alpha(3, 2)$ is much larger than the weights $\alpha(3, 0)$ and $\alpha(3, 1)$, then the decoder will pay much more attention to the encoder's output for word #2 ("soccer") than to the other two outputs.

But how do we get all these $\alpha(t, i)$ weights?

Attention Mechanisms (cont.)

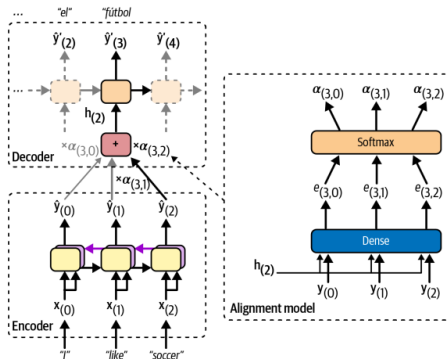


Figure 16-7. Neural machine translation using an encoder-decoder network with an attention model

Well, all these $\alpha(t, i)$ weights are generated by a small neural network called an *alignment model* (or an *attention layer*), which is trained jointly with the rest of the encoder-decoder model.

The blue **Dense** layer, which contains **only one neuron**, processes the four input vectors⁵ and outputs a score $e_{3,i}$ for each $y(i)$ for the current time step (3).

Through the **softmax** layer,

the three cores are then converted into probabilities, which are used as weights to help the decoder to predict the next word.

⁵ vector $h(2)$ is the hidden state output from the previous time step (in this case, time step 2) of the decoder; $y(0)$, $y(1)$, and $y(2)$ are three output vectors from the encoder.

Attention Mechanisms (cont.)

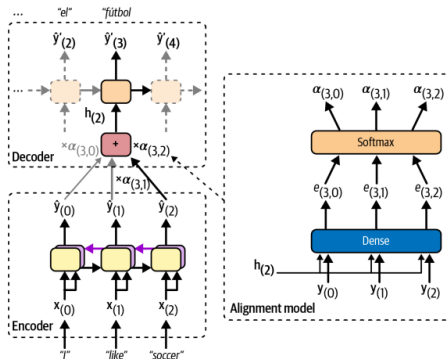


Figure 16-7. Neural machine translation using an encoder-decoder network with an attention model

Bahdanau attention. E.g., in the diagram, the Decoder has already output “**me gusta** **el**” (meaning “I like”) so it’s now expecting a noun: the word “**soccer**” is the one that best aligns with the current state, so $e_{(3,2)}$ gets a high score and thus a high probability $\alpha_{(3,2)}$ after the **softmax** function.

This particular attention mechanism is called *Bahdanau attention* (named after the 2014 paper’s first author).

Since it concatenates the output y vectors from the Encoder and the previous hidden state h from the Decoder, this attention mechanism is also called *concatenative attention* or *additive attention*.

Attention Mechanisms (cont.)

Luong attention or multiplicative attention

This is another common attention mechanism proposed shortly after, in 2015, by Luong et al⁶. The authors proposed to simply compute the **dot product** of the output vectors **y** and hidden state vector **h**, as this is often a fairly good similarity measure, and modern hardware can compute it very efficiently.

The only constraint imposed on the network is these vectors must be of the same length.

⁶ Minh-Thang Luong et al., "Effective Approaches to Attention-Based Neural Machine Translation", Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (2015): 1412–1421.

Attention Mechanisms (cont.)

Three variants of Luong attention mechanism

The Luong attention mechanism is defined as:

$$\tilde{\mathbf{h}}_{(t)} = \sum_i \alpha_{(t,i)} \mathbf{y}_{(i)}$$

with

$$\alpha_{(t,i)} = \frac{\exp(e_{(t,i)})}{\sum_{i'} \exp(e_{(t,i')})}$$

$$e_{(t,i)} = \begin{cases} \mathbf{h}_{(t)}^\top \mathbf{y}_{(i)} & \text{dot product} \\ \mathbf{h}_{(t)}^\top \mathbf{W} \mathbf{y}_{(i)} & \text{general dot product} \\ \mathbf{v}^\top \tanh(\mathbf{W} [\mathbf{h}_{(t)}; \mathbf{y}_{(i)}]) & \text{concat} \end{cases}$$

The authors compared the concat attention mechanism against the two dot product attention mechanisms and found that the former was outperformed by the two latter mechanisms. Because of that, the concat variant of Luong attention mechanism is less used now.

Attention Mechanisms (cont.)

Implementing attention mechanisms using Keras

Keras provides a `tf.keras.layers.Attention` layer for Luong attention, and an `AdditiveAttention` layer for Bahdanau attention. Since we will need to pass all the encoder's outputs to the Attention layer, we need to set `return_sequences=True` when creating the encoder:

```
encoder = tf.keras.layers.Bidirectional(  
    tf.keras.layers.LSTM(256, return_sequences=True, return_state=True))
```

For simplicity, we can pass the decoder's outputs instead of its states to the attention layer: in practice this works well too, and it's much easier to code, as shown below:

```
attention_layer = tf.keras.layers.Attention() # default: score_mode="dot"  
attention_outputs = attention_layer([decoder_outputs, encoder_outputs])  
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")  
Y_proba = output_layer(attention_outputs)
```

By default, the *dot product* variant is used in the `Attention` layer. To use the *concat* variant, set `score_mode="concat"`.

Attention Mechanisms (cont.)

The concepts of “keys”, “values”, and “queries”

Consider the sentence “I like soccer” again. Suppose that the encoder managed to understand the word “I” is the subject and the word “like” is the verb in the encoding process. Now suppose that the decoder has translated the subject “I” into Spanish and it should translate the verb next. For this, it needs to fetch the verb from the input sentence. This is analogous to a *dictionary lookup* – it is as if the encoder had created a dictionary {“subject”: “They”, “verb”: “played”, ...} and the decoder wanted to look up the **value** that corresponds to the **key** “verb”.

The outputs from the encoder are the **keys** and, optionally, the **values**; the outputs from the decoder are the **queries**.

Attention Mechanisms (cont.)

The concepts of “keys”, “values”, and “queries” (cont.)

The `attention_layer` expects 2 to 3 inputs. If the *values* input is not provided, then it is treated as equal to the *keys* input.

The line (see slide 26)

```
attention_outputs = attention_layer([decoder_outputs, encoder_outputs])
```

can therefore be interpreted as

`attention_outputs = attention_layer([queries, keys])`

The output `attention_outputs` is then converted into an array of probabilities (variable `Y_proba` on slide 26) through the `Dense` layer and the `softmax` activation. The length of the `Y_proba` array corresponds to the vocabulary size of the target language (in our example, Spanish).

These concepts are all represented as vectors in the model and are incorporated extensively in the `transformer` architecture.

The Original Transformer Architecture

In the 2017 groundbreaking paper “**Attention Is All You Need**”⁷ from Google, the authors proposed an architecture called the *transformer*, which significantly improved the state-of-the-art in NMT without using any recurrent or convolutional layers, just attention mechanisms (plus embedding layers, dense layers, etc).

Because the model is not recurrent, it doesn't suffer as much from the vanishing or exploding gradients problems as RNNs. It can be trained in fewer steps, it's easier to parallelize across multiple GPUs, and it can better capture long-range patterns than RNNs.

⁷ Ashish Vaswani et al., “Attention Is All You Need”, Proceedings of the 31st International Conference on Neural Information Processing Systems (2017): 6000–6010.

The Original Transformer Architecture (cont.)

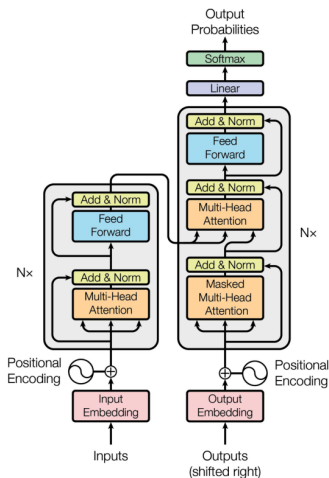


Figure 16-8. The original 2017 transformer architecture

In the diagram, the left part is the *encoder*; the right part is the *decoder*. Each embedding layer outputs a 3D tensor of shape [batch size, sequence length, embedding size]. After that, the tensors are gradually transformed as they flow through the transformer, but their shape remains the same. In the 2017 paper, the authors set $N = 6$.

There are two embedding layers (pink boxes). There are some skip connections followed by Addition and Normalisation layers. The Feed Forward modules (light blue boxes) comprise two Dense layers each: the first layer uses ReLU; the second layer has no activation function. Since all these layers treat each word independently from the others, we need a layer that captures the context and relationships among words. This is the role of the *multi-header attention* layer in the network.

The Original Transformer Architecture (cont.)

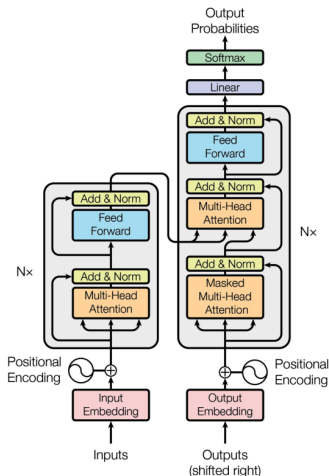


Figure 16-8. The original 2017 transformer architecture

- The first two arrows going into each multi-head attention layer represent the *keys* and *values*, and the third arrow represents the *queries*.
- The encoder's *multi-head attention* layer updates each word representation by attending to (i.e., paying attention to) all other words in the same sentence.
- The decoder's *masked multi-head attention* layer does the same thing, but when it processes a word, it doesn't attend to words located after it: it's a causal layer. This is done by the additional *masking* operation.
- The *positional encodings* are dense vectors (much like word embeddings) that represent the position of each word in the sentence.

The Original Transformer Architecture (cont.)

There are two key components in the Transformer architecture: (i) *the Positional Encoding layer*, (ii) *the Multi-Head Attention mechanism*.

Positional encodings

A *positional encoding* is a dense vector that encodes the position of a word within a sentence: the i^{th} positional encoding is added to the word embedding of the i^{th} word in the sentence. The easiest way to implement this is to use an Embedding layer and make it encode all the positions from 0 to the maximum sequence length in the batch, then add the result to the word embeddings.

The authors of the *transformer* paper chose to use fixed positional encodings, based on the sine and cosine functions at different frequencies. Their positional encoding matrix $\mathbf{P}$ is defined as:

$$\mathbf{P}_{p,i} = \begin{cases} \sin(p/1000^{i/d}) & \text{if } i \text{ is even} \\ \cos(p/1000^{(i-1)/d}) & \text{if } i \text{ is odd} \end{cases}$$

where $\mathbf{P}_{p,i}$ denotes the positional encoding for the i^{th} component of the embedding vector for the word located at the p^{th} position in the sentence; and d denotes the dimension of the *keys*, *values*, and *queries* vectors.

The Original Transformer Architecture (cont.)

Positional encodings (cont.)

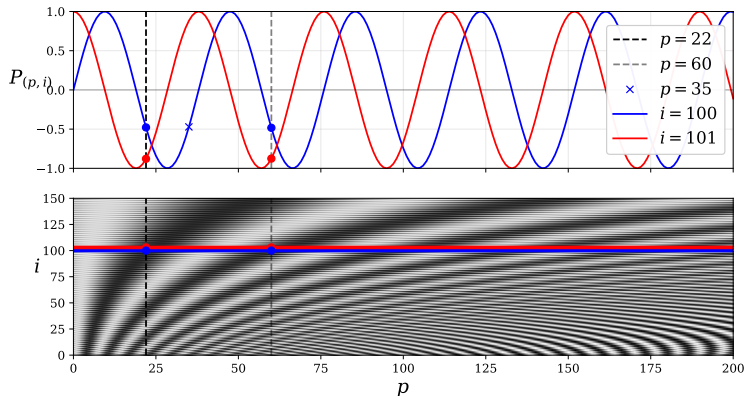


Figure 16-9. Sine/cosine positional encoding matrix (transposed, bottom) with a focus on two values of i (top)

After these positional encodings are added to the word embeddings, the rest of the model has access to the absolute position of each word in the sentence because each position has a unique positional encoding.

The Original Transformer Architecture (cont.)

Positional encodings (cont.)

TensorFlow provides a `tfm.vision.layers.PositionalEncoding` layer. Using the `tfm` library requires additional installation `tensorflow_models`⁸. However, it is not too hard to create one yourself.

For efficiency reasons, we precompute the positional encoding matrix in the constructor. The `call()` method just truncates this encoding matrix to the maximum length of the input sequences, and it adds them to the inputs. We also set `supports_masking=True` to propagate the input's automatic mask to the next layer:

⁸Installation can be done by running the command: `pip install tf-models-official`. **Warning:** this is probably a very large library and is not easy to install.

The Original Transformer Architecture (cont.)

Positional encodings (cont.)

```
class PositionalEncoding(tf.keras.layers.Layer):
    def __init__(self, max_length, embed_size, dtype=tf.float32, **kwargs):
        super().__init__(dtype=dtype, **kwargs)
        assert embed_size % 2 == 0, "embed_size must be even"
        p, i = np.meshgrid(np.arange(max_length),
                            2 * np.arange(embed_size // 2))
        pos_emb = np.empty((1, max_length, embed_size))
        pos_emb[0, :, ::2] = np.sin(p / 10_000 ** (i / embed_size)).T
        pos_emb[0, :, 1::2] = np.cos(p / 10_000 ** (i / embed_size)).T
        self.pos_encodings = tf.constant(pos_emb.astype(self.dtype))
        self.supports_masking = True

    def call(self, inputs):
        batch_max_length = tf.shape(inputs)[1]
        return inputs + self.pos_encodings[:, :batch_max_length]
```

The Original Transformer Architecture (cont.)

Positional encodings (cont.)

After that, adding the positional encoding to the encoder's outputs is straightforward:

```
pos_embed_layer = PositionalEncoding(max_length, embed_size)
encoder_in = pos_embed_layer(encoder_embeddings)
decoder_in = pos_embed_layer(decoder_embeddings)
```

The code for the embedding on the decoder part of the network is similar.

The Original Transformer Architecture (cont.)

Multi-head attention

Now let's look deeper into the *multi-head attention* layer, the heart of the transformer model.

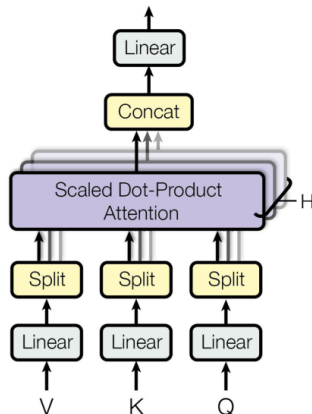


Figure 16-10. Multi-head attention layer architecture

To understand how a multi-head attention layer works, we must first understand the *scaled dot-product attention* layer, which outputs:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{keys}}}} \right) \mathbf{V}$$

where

- $\mathbf{Q}$ (shape: $[n_{\text{queries}}, d_{\text{keys}}]$; *query* matrix) contains one row per query;
- $\mathbf{K}$ (shape: $[n_{\text{keys}}, d_{\text{keys}}]$; *key* matrix) contains one row per key;
- $\mathbf{V}$ (shape: $[n_{\text{keys}}, d_{\text{values}}]$; *value* matrix) contains one row per value.

The Original Transformer Architecture (cont.)

Multi-head attention (cont.)

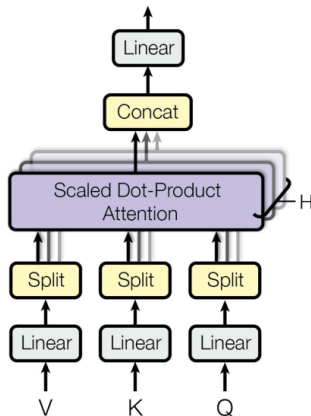


Figure 16-10. Multi-head attention layer architecture

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_{\text{keys}}}} \right) \mathbf{V}$$

So,

- $\mathbf{Q}\mathbf{K}^{\top}$ (shape: $[n_{\text{queries}}, n_{\text{keys}}]$) contains one similarity score for each query/key pair.
- The factor $1/d_{\text{keys}}$ scales down the similarity scores to avoid saturating the softmax function, which would lead to tiny gradients.
- The output from the softmax function has the same shape (also $[n_{\text{queries}}, n_{\text{keys}}]$) but the numbers for each row add up to 1.
- The output shape of $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ is therefore $[n_{\text{queries}}, d_{\text{values}}]$ where each row represents the query result (a weighted sum of the values).

The Original Transformer Architecture (cont.)

Multi-head attention (cont.)

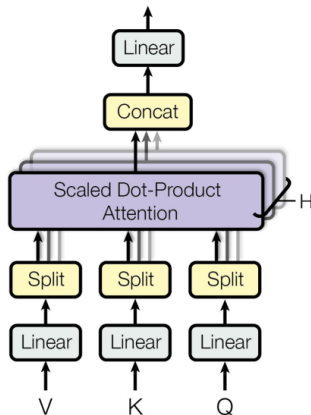


Figure 16-10. Multi-head attention layer architecture

- In the training stage, the data going into the network and the multi-head attention layer are 3D tensor (axis 0 is the minibatch size). All the matrix multiplications can be done efficiently in Python.
- Notice that after V , K , and Q are individually transformed by the *Linear* layers (see diagram), they are split (yellow box) and later on concatenated back (top yellow box). The separate parts going through the *Scaled Dot-Product Attention* layer allow different characters of the words to be captured (e.g., not only the word “like” is a verb, it is also in present tense).
- The number of parts that V , K , and Q are split is determined by the value of H (*number of heads*).

The Original Transformer Architecture (cont.)

Multi-head attention (cont.)

- The multi-head attention layer in the **encoder** is a *self-attention* layer as it updates each word representation by attending to (i.e., paying attention to) all other words in the same sentence.
- The decoder's upper multi-head attention layer is where the decoder pays attention to the words in the English sentence. This is called cross-attention, not self-attention in this case. For example, the decoder will probably pay close attention to the word "**soccer**" when it processes the word "**el**" and transforms its representation into a representation of the word "**fútbol**".

The Original Transformer Architecture (cont.)

Summary

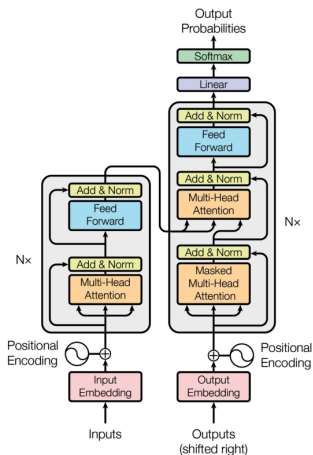
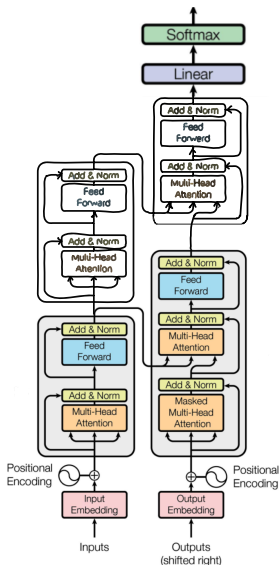


Figure 16-8. The original 2017 transformer architecture

- The decoder uses a starting token (e.g., **<SOS>**) as the prompt. After the first word is predicted, the masking on the first target word is removed and **the entire network is run again to predict the second word using the first word**. This process is repeated until the network outputs (or learns to output) the ending token (**<EOS>**).
- The green **softmax** layer have many neurons (corresponding to the vocabulary size of the target language). **Beam search** (not covered in the unit) is often used to help the network focus on those words having high probabilities.

The Original Transformer Architecture (cont.)



Summary (cont.)

- At the inference stage, there is nothing going into the decoder part except for the $\langle \text{SOS} \rangle$ prompt. After the first word is predicted, it is used as input to the decoder. After the second word is predicted, it is appended to the first word. So, the input to the decoder has two words. This process is repeated until the $\langle \text{EOS} \rangle$ is output.

The transformer architecture with $N = 2$

The Original Transformer Architecture (cont.)

Implementation using Keras

Keras includes a `tf.keras.layers.MultiHeadAttention` layer, so we now have everything we need to build the rest of the transformer.

```
N = 2          # instead of 6
num_heads = 8
dropout_rate = 0.1
n_units = 128 # for the first Dense layer in each Feed Forward block
encoder_pad_mask = tf.math.not_equal(encoder_input_ids, 0)[: , tf.newaxis]
Z = encoder_in
for _ in range(N):
    skip = Z
    attn_layer = tf.keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_size, dropout=dropout_rate)
    Z = attn_layer(Z, value=Z, attention_mask=encoder_pad_mask)

# ... add in other layers as shown in Figure 16-8
```

Vision Transformers

Image captioning using visual attention

One of the first applications of attention mechanisms beyond NMT was in generating *image captions* using *visual attention*⁹: a convolutional neural network first processes the image and outputs some feature maps, then a decoder RNN equipped with an attention mechanism generates the caption, one word at a time.

At each decoder time step (i.e., each word), the decoder uses the attention model to focus on just the right part of the image.

⁹ Kelvin Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", Proceedings of the 32nd International Conference on Machine Learning (2015): 2048-2057.

Vision Transformers (cont.)

Image captioning using visual attention (cont.)

For example, in Figure 16-12, the model generated the caption “**A woman is throwing a frisbee in a park**”, and you can see what part of the input image the decoder focused its attention on when it was about to output the word “**frisbee**”: clearly, most of its attention was focused on the frisbee.



Figure 16-12. Visual attention: an input image (left) and the model's focus before producing the word “frisbee” (right)

Vision Transformers (cont.)

The original vision transformer

In October 2020, a team of Google researchers released a paper¹⁰ that introduced a fully transformer-based vision model, called a *vision transformer* (ViT). The idea is surprisingly simple:

- Just chop the image into little 16×16 squares, and treat the sequence of squares as if it were a sequence of word representations.
- The squares are first flattened into $16 \times 16 \times 3 = 768$ -dimensional vectors – the value 3 is for the RGB colour channels.
- These vectors then go through a linear layer that transforms them but retains their dimensionality. The resulting sequence of vectors can be treated as a sequence of word embeddings: i.e., adding positional embeddings, and passing the result to the transformer.

This model beat the state of the art on **ImageNet** image classification, but required over 300 million additional images for training. This makes sense since transformers don't have as many inductive biases as CNNs, so they need extra data just to learn things that CNNs implicitly assume.

¹⁰ Alexey Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale", arXiv preprint arxiv:2010.11929 (2020).

Vision Transformers (cont.)

(Materials from this slide onward are not covered in the exam)

Other related papers

- The research field quickly expanded. Just two months later, a paper¹¹ that introduced *data-efficient image transformers* (DeiT_s) was released from a team of Facebook, where the authors used a *distillation* technique to reduce the number of training images required.
- Then in March 2021, DeepMind released an important paper¹² that introduced the *Perceiver* architecture – a multimodal transformer, which can handle text, images, audio, etc.
- Just a month later, Caron et al. introduced *DINO*¹³, an impressive vision transformer trained entirely without labels and capable of high-accuracy semantic segmentation.

¹¹ Hugo Touvron et al., “Training Data-Efficient Image Transformers & Distillation Through Attention”, arXiv preprint arxiv:2012.12877 (2020).

¹² Andrew Jaegle et al., “Perceiver: General Perception with Iterative Attention”, arXiv preprint arxiv:2103.03206 (2021).

¹³ Mathilde Caron et al., “Emerging Properties in Self-Supervised Vision Transformers”, arXiv preprint arxiv:2104.14294 (2021).

Vision Transformers (cont.)

Other related papers (cont.)

- A 2021 paper¹⁴ from Google showed how to scale ViTs up or down, depending on the amount of data. They managed to create a huge 2 billion parameter model that reached over 90.4% top-1 accuracy on ImageNet. Conversely, they also trained a scaled-down model that reached over 84.8% top-1 accuracy on ImageNet, using only 10,000 images: that's just 10 images per class!
- In April 2022, DeepMind released the *Flamingo paper*¹⁵, which introduced a family of models pretrained on a wide variety of tasks across multiple modalities, including text, images, and videos.
- Then in May 2022, DeepMind introduced *GATO*¹⁶, a multimodal model that can be used as a policy for a reinforcement learning agent.

¹⁴ Xiaohua Zhai et al., "Scaling Vision Transformers", arXiv preprint arxiv:2106.04560v1 (2021).

¹⁵ Jean-Baptiste Alayrac et al., "Flamingo: a Visual Language Model for Few-Shot Learning", arXiv preprint arxiv:2204.14198 (2022).

¹⁶ Scott Reed et al., "A Generalist Agent", arXiv preprint arxiv:2205.06175 (2022).

Hugging Face's Transformers Library

It's impossible to talk about transformers today without mentioning **Hugging Face**, an AI company that has built a whole ecosystem of easy-to-use open source tools for NLP, vision, and beyond.

The central component of their ecosystem is the Transformers library, which allows you to easily download a pretrained model, including its corresponding tokenizer, and then fine-tune it on your own dataset, if needed. Plus, the library supports TensorFlow, PyTorch, and JAX (with the Flax library).

Hugging Face has also built a Datasets library (similar to the TensorFlow Datasets) that you can use to easily download a standard dataset (such as IMDb) or a custom one, and use it to fine-tune your model.

(Unfortunately, we don't have room to include this library in the unit)

Summary

- Understand the encoder-decoder networks for neural machine translation
- Understand bidirectional RNNs and LSTMs
- Understand attention mechanisms and the differences between additive attention and multiplicative attention
- Know how to implement attention mechanisms using Keras
- Understand visual attention
- Understand transformers and vision transformers

For the next topic...

Read up to Chapter 18 on **Reinforcement Learning**.

And that's all for this lecture.

Have a good week.